PLOT-TWIST GENERATOR

Comparing Vanilla RNN and LSTM Language Models

Course: Natural Language Processing

Assignment: Group Assignment 2 — Question 4

Dataset: TinyStories (selected 22,000-story stream subset)

Models: Vanilla RNN and LSTM | Seed: 42

Submission status: experiments and outputs are included. Genuine human ratings are still required; blank rating forms are provided and are not represented as completed evaluation.

Abstract

This project implements two recurrent neural language models—a vanilla Recurrent Neural Network (RNN) and a Long Short-Term Memory (LSTM)—to continue short stories from prompts. Both models use the same word-level vocabulary and story-level train/validation/test split. They are trained with next-token cross-entropy and compared using held-out test perplexity, lexical diversity, 3-gram repetition, and qualitative inspection of a Plot-Twist Challenge. The recorded LSTM checkpoint achieves lower test perplexity (13.40 versus 16.88), higher Distinct-1/Distinct-2, and a lower mean 3-gram repetition rate than the RNN. However, both models often drift away from the unusual prompt and produce generic children's-story events rather than meaningful twists. The results demonstrate that stronger next-token prediction and higher lexical diversity do not guarantee coherent narrative planning. A blinded human-rating set is prepared, but its ratings remain blank pending genuine evaluation.

1. Objectives

- Prepare a consistent tokenization and next-token prediction pipeline.
- Train a vanilla RNN and an LSTM under comparable settings.
- Compare validation/test perplexity, diversity, repetition, and output length.
- Implement greedy decoding, temperature sampling, and top-k sampling.
- Test at least five twist-oriented openings and inspect failures rather than omitting them.
- Prepare a blinded human-evaluation form; do not treat uncollected ratings as results.

2. Dataset and preprocessing

The code uses the TinyStories dataset (`roneneldan/TinyStories`) through the Hugging Face `datasets` library. It takes the first 22,000 stories from the training stream, shuffles them with seed 42, and divides them at the story level into 20,000 training stories, 1,000 validation stories, and 1,000 test stories. The raw dataset and processed token-ID splits are intentionally excluded from the ZIP; rerunning preprocessing requires internet access to download the dataset.

Setting	Implementation
Tokenization	Lowercased word-level regex; curly quotation marks normalized
Vocabulary	Built from training split only; minimum frequency 5; maximum 10,000 entries
Saved vocabulary	6,695 tokens, including four special tokens
Special tokens	<pad>=0, <unk>=1, <bos>=2, <eos>=3
Sequence length	Maximum 200 tokens; longer stories truncated
Story boundaries	Each story encoded independently; no concatenation across stories
Padding and loss	Batch padding; cross-entropy ignores <pad> targets

Split	Stories	Tokens	Mean length	UNK rate	Truncated
Train	20,000	3,483,783	174.2	0.22%	32.7%
Validation	1,000	173,213	173.2	0.34%	30.7%
Test	1,000	174,787	174.8	0.34%	32.2%

A limitation of this preprocessing run is that roughly 31–33% of stories are truncated at 200 tokens; when truncation occurs, an end-of-sequence token is not appended. The recorded vocabulary and generated samples also reveal a small number of garbled symbols (for example, "€" and "™"), indicating that text cleaning could be improved.

3. Model architecture and training

Both models use an embedding layer, one recurrent layer, and a linear projection to vocabulary logits. The RNN uses `torch.nn.RNN`; the LSTM uses `torch.nn.LSTM`. Both are trained to predict the next token from the preceding token sequence using cross-entropy loss, with padding excluded from the loss. Model comparison is controlled by using the same vocabulary, split, embedding/hidden dimensions, batch size, optimizer, and seed.

Setting	Value
Embedding / hidden dimension	256 / 512

Recurrent layers	1
Dropout	0.2
Optimizer / learning rate	Adam / 0.001
Batch size	64
Gradient clipping	1.0
Maximum epochs / patience	10 / 3
Random seed	42
Device	Original run hardware not recorded in the available artifacts

Both recorded training histories contain 10 epochs, and validation loss was still decreasing at the final epoch. Early stopping therefore did not trigger, and both checkpoints should be viewed as potentially undertrained. The loss curves are provided in `artifacts/loss_curves.png`; detailed epoch histories are saved in `artifacts/rnn_history.json` and `artifacts/lstm_history.json`.

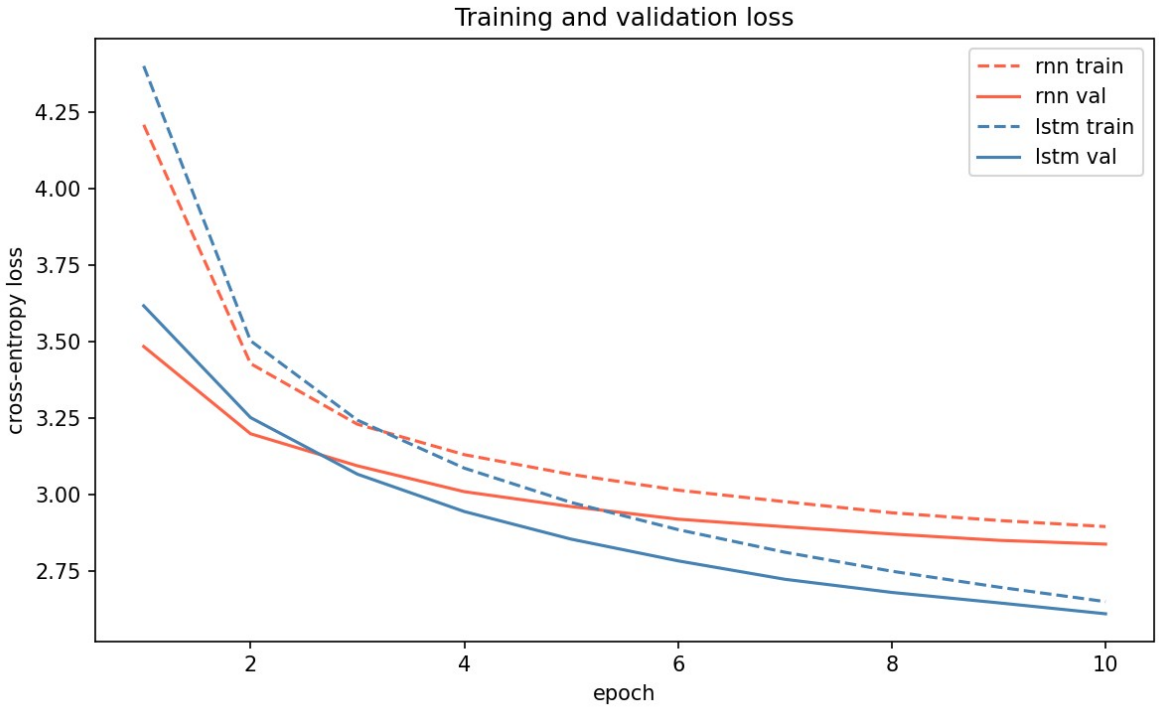


Figure 1. Recorded training and validation loss curves for the two models.

4. Quantitative results

Metric	Vanilla RNN	LSTM
Trainable parameters	5,542,695	6,725,415
Training time	343.74 s	473.14 s
Best validation perplexity	17.07	13.59
Test loss	2.8261	2.5955
Test perplexity	16.88	13.40
Distinct-1	0.189	0.205
Distinct-2	0.480	0.544
Mean 3-gram repetition rate	0.1033	0.0837
Mean output length, including prompt	70.77 words	78.91 words
Generation samples for pooled metrics	35	35

Perplexity is exp(mean per-token cross-entropy); lower is better for next-token prediction on the same held-out data. Distinct-1/2 are unique unigrams/bigrams divided by total unigrams/bigrams. Repetition rate is 1 − (unique 3-grams / total 3-grams), averaged per output. The pooled metrics use 35 generated outputs per model from seven general prompts and five decoding settings; the prompt is included in these pooled texts. Under the project's thresholds, no outputs were under 20 words and none reached 180 words. The latter count is not very informative because generation is capped at 80 new tokens.

The LSTM has about 21.3% more parameters than the RNN and took about 37.6% longer in the recorded run. Despite that cost, it improved test perplexity from 16.88 to 13.40, raised Distinct-2 from 0.480 to 0.544, and reduced mean 3-gram repetition from 0.1033 to 0.0837. These are consistent improvements on the measured tasks, but they do not directly measure plot quality.

5. Generation strategies and Plot-Twist Challenge

Strategy	Rule	Observed trade-off
Greedy	Choose the highest-probability next token each step.	Deterministic and often predictable; can fall into repetitive loops.
Temperature sampling	Divide logits by T, then sample; tested T=0.5, 0.8, 1.2.	Higher T increases variation but can produce abrupt or incoherent transitions.
Top-k sampling	Sample among the k most likely tokens; k=10.	Restricts unlikely tokens but does not enforce character or plot consistency.

The Plot-Twist Challenge contains 84 saved records: seven prompts × two models × three strategies × two sample slots. It covers an exam paper with tomorrow's answers, a robot receiving a message from its future self, a detective solving a future murder, a village forgetting the previous day, a letter from 100 years in the future, a new student who knows everyone's secrets, and the moon moving closer to a village. Greedy decoding is deterministic for the same prompt and checkpoint, so its paired records are duplicates. Outputs are preserved without rewriting in `results/plot_twist_challenge.json`.

6. Qualitative analysis

6.1 RNN greedy: prompt drift and repetition

Prompt: "A detective solved a murder that had not happened yet."

Recorded continuation (verbatim excerpt): "he was very excited and wanted to play with it . he asked his mom , " what is that ? " the bird said , " i am a little girl . i am a little girl . i am a little girl . i am a little girl . i am a little girl . i am a little girl . i am a little girl . i am a little girl"

The output quickly abandons the detective and the future-murder premise. The shift to a bird and a little girl is unmotivated, and the repeated sentence is a clear degeneration loop. No investigation or twist develops.

6.2 RNN temperature: more variety without relevance

Prompt: "A student found an exam paper containing tomorrow's answers."

Recorded continuation: "he was a very powerful puppy who was very competitive and he loved to explore the world . one day , he saw a big lake with many things in a cage . the cat wanted to eat the food but it was too late . timmy and his friends were excited to have found their puppy ."

The sample includes more entities and events than the greedy loop, but the exam paper and future answers disappear. The puppy, lake, cat, and Timmy do not form a coherent chain tied to the prompt. More varied text is not necessarily a better continuation.

6.3 LSTM greedy: locally readable but generic

Prompt: "A student found an exam paper containing tomorrow's answers."

Recorded continuation: "the little girl was so excited ! she ran around the house , looking for something to eat . she saw a big , green frog . she was so excited ! she ran to the kitchen and opened it . she was so excited ! she ran around the house and found a big , shiny object ."

The actions have some local continuity, but the continuation ignores the exam premise and repeats "she was so excited". This is a useful example of how locally readable sentences can still fail the story task.

6.4 LSTM temperature: entity drift and malformed text

Prompt: "An old robot received a message from its future self."

Recorded continuation: “she was so excited that she thanked her mom for making her salad for her special day . she said goodbye to the puppy and to the big stack . the little girl was so happy . she shouted the dress - <unk> , and all the books and showed her how to make her own secrets .”

The robot and future message disappear; the output jumps among a girl, salad, puppy, books, and a dress, and contains an unresolved <unk> token. Sampling creates lexical variety but not a consistent narrative.

6.5 Main failure patterns

- Prompt drift: the model falls back to common TinyStories-style scenes rather than using the distinctive premise.
- Character and entity drift: robots, detectives, students, animals, and children change without explanation.
- Repetition: greedy decoding can repeat clauses or sentences, especially for the RNN.
- Weak causal structure: actions and objects appear in sequence without a meaningful relationship.
- Data/tokenization artifacts: <unk> and occasional garbled symbols appear in generated text.
- No reliable twist: the model rarely turns the opening premise into a surprising but consistent development.

Possible future remedies include more training data, longer training with monitored validation loss, improved cleaning and tokenization, repetition penalties or n-gram blocking, and explicit story-planning or stronger prompt-conditioning methods. These interventions were not tested in the recorded experiments and are recommendations for future work, not measured improvements.

7. Human evaluation

The assignment requests at least ten generated continuations rated blindly on coherence, grammar, prompt relevance, and creativity/surprise using a 1–5 scale; three evaluators are recommended where feasible. The package now includes a 30-sample blinded sheet, a 90-row three-rater template, a separate sample-to-model key, and instructions. The score fields are blank because genuine evaluator responses were not available when this report was prepared. No human means, inter-rater agreement, or human preference is claimed.

Before final submission, have real evaluators independently rate the blinded samples. Keep `results/human_eval_key.csv` away from evaluators until scoring is finished. Then calculate per-criterion averages, report the number of raters, discuss disagreements, and compare those ratings with the automatic metrics. This is a rubric-critical outstanding step; the blank form is preparation for the study, not evidence that it has been completed.

8. Discussion: answering the assignment questions

RNN versus LSTM. The LSTM performed better on held-out next-token prediction and had lower pooled repetition, at the cost of more parameters and training time. Its gating mechanism is designed to retain information over longer sequences, but the outputs show that this did not solve narrative planning.

Which generates more coherent, less repetitive text? The recorded LSTM metrics favor it, and representative samples can be locally more readable. However, both models often ignore the prompt; without genuine human ratings, a broad human-perceived coherence claim cannot be made.

Temperature effects. Lower temperature concentrates probability on likely tokens; higher temperature increases variation. The recorded high-temperature examples can drift or become malformed. Temperature 0.8 and top-k are reasonable settings to inspect, but neither guarantees relevance.

Do diversity scores guarantee better stories? No. Diversity counts lexical variety, not meaning, causality, prompt adherence, or surprise. A continuation can be varied and still incoherent.

Does lower perplexity imply better storytelling? No. LSTM perplexity is lower, but both models fail to develop many challenge premises. Perplexity evaluates next-token prediction on held-out text rather than narrative quality directly.

Most common failures and potential fixes. Prompt drift, repetitive loops, character inconsistency, abrupt transitions, and tokenization artifacts. Potential fixes are listed above but were not tested.

Limitations. A 20k/1k/1k story split, one random seed, a 10-epoch cap, truncation of many stories, word-level tokenization, seven challenge prompts, capped generations, and no completed human ratings. Results are therefore exploratory and should not be generalized beyond this setup.

9. Reproducibility and submission contents

The archive includes source scripts, model checkpoints, the vocabulary, data statistics, loss histories and plot, saved quantitative metrics, general generation outputs, challenge outputs, and human-rating materials. It excludes the raw dataset and processed split tensors to avoid bundling the dataset subset. Preprocessing and test-set reevaluation require downloading TinyStories again.

- Install dependencies: `pip install -r requirements.txt`.
- Download/preprocess data: `python -m src.preprocess`.
- Train either model: `python -m src.train --model rnn` or `python -m src.train --model lstm`.
- Regenerate loss plot: `python -m src.plot_curves`.
- Generate continuations: `python -m src.generate --model lstm --strategy temperature --temperature 0.8`.
- Run the challenge: `python -m src.challenge`.
- Recompute metrics: `python -m src.metrics` (requires processed train/validation/test splits).
- Create blinded evaluation forms: `python -m src.human_eval`.

The run hardware and exact installed library versions were not preserved in the supplied artifacts; training times are therefore reported as recorded, not as a portable runtime guarantee.

10. Conclusion

In this run, the LSTM is the stronger next-token language model: it achieves lower test perplexity, somewhat higher lexical diversity, and lower mean 3-gram repetition than the vanilla RNN. Yet both models struggle with the core creative task. Their continuations frequently replace the challenge premise with generic children's-story events, and greedy decoding can loop. The central conclusion is that next-token modeling metrics and lexical diversity are useful but insufficient measures of storytelling ability. A genuine blinded human evaluation remains necessary before the final submission can claim to have completed the assignment's human-evaluation requirement.